

1.1. Data Download

The Minimus stores recorded seismic data in miniSEED format on its dual microSD cards. Data can be downloaded using the web interface, an automated script, or by removing the external microSD card and copying files manually. Use the method most suitable for your workflow.

1.1.1. Download via Web Interface

The **Storage** page of the Minimus web interface provides access to miniSEED, metadata and log files.

**Note:** To ensure the most recent data has been written to the SD cards, click the **Flush Data** button before downloading files.

SD Card control

Flush data	Unmount Cards	Quickformat Cards	Fullformat Cards
		Regenerate XMLs	Full-format is not in

To download files:

1. Open the Minimus web interface, login and navigate to the **Storage** page.
2. Select one or more files by ticking the **check-boxes** on the left of each file link.
3. Click **Download Selected Files** to download them to your local device.

☒	00A555_S0SeisEA_00200_00021.mseed	134217728	2026-02-12 20:52:22.000000000
☒	00A555_S0SeisNA_00200_00022.mseed	26451968	2026-02-13 11:50:21.390000000
☒	00A555_S0SeisZA_00200_00023.mseed	26058752	2026-02-13 11:49:54.230000000
☒	00A555_S0SeisEA_00200_00024.mseed	23048192	2026-02-13 11:50:10.505000000
☐	status.log	1310574	2026-02-13 11:46:39.000000000
☐	system.log	111107	2026-02-13 11:47:29.000000000
☐	init.log	111107	2026-02-13 11:47:29.000000000
☐	Sensor0.xml	857842	2026-02-13 09:16:03.000000000
☐	Sensor1.xml	514048	2026-02-13 09:16:11.000000000
☐	Sensor2.xml	513481	2026-02-13 09:16:18.000000000
☐	Sensor3.xml	513481	2026-02-13 09:16:25.000000000
☐	Sensor4.xml	513481	2026-02-13 09:16:34.000000000
☐	Sensor5.xml	513481	2026-02-13 09:16:42.000000000
☐	Sensor6.xml	513481	2026-02-13 09:16:49.000000000
☐	Sensor7.xml	513481	2026-02-13 09:16:55.000000000
☐	table_of_events.bin	1024	2017-01-01 03:00:00.000000000

Download selected files **Download all files**

1.1.1.1. Download via Time Selection

The “Channel data download by time selection” section allows downloading a specific channel for a designated time interval. This feature generates a new miniSEED file containing only the selected period.

To download by time selection:

1. Select the desired channel from the drop-down menu.
2. Specify the start and end date using the calendar controls.

3. Set the start and end time in 24-hour format.
4. Click **Download** to generate and save the miniSEED file.

Channel data download by time selection			
Channel:	DG.0A555.0K.HDF [S0SeisXA] ▼	From:	01/01/2026 00:00 To: 02/01/2026 06:00 Download

1.1.2. Download via Bash Script

A Bash script is provided to download data directly from the Minimus over a network connection.

The script uses HTTP requests to retrieve data stored on the device and can be used to download either all available files or a specific time window. It is intended to be run on a Linux system, or on Windows using the Windows Subsystem for Linux (WSL).

Two modes of operation are supported:

- bulk download (all stored files)
- time-based download (specific time window)

Example Script:

```
#!/bin/bash

# Minimus Data Download Script

#

# This script supports two modes:

#   1. Bulk download (all files from storage)

#   2. Time-based download (specific time window, miniSEED output)

#

# Usage:

#   ./download.sh <device_ip> [<date YYYY-MM-DD>] [<start_hour 0-23>]

#   [<number_of_hours>]

#

# Examples:

#   Bulk download:

#     ./download.sh 192.168.0.10

#

#   Time-based download (1 hour from 14:00 on a given date):

#     ./download.sh 192.168.0.10 2025-02-21 14 1

set -e # Exit immediately if a command fails
```

```

#####
# Behaviour Options
#####

# Set to "true" to combine channels into a single file
COMBINE_CHANNELS=false


#####
# Validate Inputs
#####

if [ "$#" -lt 1 ]; then
    echo "Usage: $0 <device_ip> [<date YYYY-MM-DD>] [<start_hour 0-23>]
[<hours>]"
    exit 1
fi


DEVICE_IP=$1
DATE=$2
START_HOUR=$3
HOURS=$4


#####
# Prepare Output Directory
#####

# Data will be saved into a directory named after the device IP and current
data time
OUTPUT_DIR="data_${DEVICE_IP}_${date +%Y%m%d_%H%M%S}"
mkdir -p "$OUTPUT_DIR"


#####
# Mode 1: Bulk Download
#####

# If no date is provided, retrieve all files from the device storage
if [ -z "$DATE" ]; then
    echo "Starting bulk download from ${DEVICE_IP}..."

    cd "$OUTPUT_DIR"

```

```

# -r : recursive download
# -np: do not follow parent directories
# Adjust the URL if required for your device firmware
wget -r -np "http://${DEVICE_IP}/tab9.html"

cd ..

echo "Bulk download complete. Files saved in ${OUTPUT_DIR}"
exit 0
fi

#####
# Mode 2: Time-Based Download
#####
# This mode retrieves data for a specified time window

if [ -z "$START_HOUR" ] || [ -z "$HOURS" ]; then
    echo "Error: Time-based download requires <date>, <start_hour>, and <hours>"
    exit 1
fi

echo "Preparing time-based download..."

#####
# Convert Time to UNIX Timestamps
#####
# The device API requires time in UNIX format (seconds since epoch)
if ! START_UNIX=$(date -d "${DATE} ${START_HOUR}:00:00" +%s 2>/dev/null);
then
    echo "Error: Invalid date or time format"
    exit 1
fi
END_UNIX=$((START_UNIX + 3600 * $HOURS))

echo "Start time (UNIX): ${START_UNIX}"
echo "End time (UNIX): ${END_UNIX}"

```

```

#####
# Channel Configuration
#####

# These values must match the device configuration.
# They can be found on the web interface (Data Stream or Status page).

NET_CODE="XX" # Replace with Network Code (Setup Page)
STATION_CODE="XXXX" # Replace with actual station code (Setup page)
LOCATION_CODE="00" # Replace with Location Code (Data Record page)

# Typical 3-component seismic channels, Replace with actual channel code
(Data Record Page)
CHANNELS=("HNZ" "HNN" "HNE") # Replace with Channel Codes (Data Record page)

cd "$OUTPUT_DIR"

#####
# Download Each Channel
#####
for CHANNEL in "${CHANNELS[@]}"; do
    OUTPUT_FILE="${CHANNEL}.mseed"

    URL="http://${DEVICE_IP}/data?channel=${NET_CODE}.${STATION_CODE}.${LOCATION_CODE}.${CHANNEL}&from=${START_UNIX}&to=${END_UNIX}"

    echo "Downloading ${CHANNEL}..."
    echo "URL: ${URL}"

    wget -q -O "${OUTPUT_FILE}" "${URL}"
done

#####
# Combine Channels (Optional)
#####
if [ "$COMBINE_CHANNELS" = true ]; then

```

```

DATE_STRING=$(date -d "@${START_UNIX}" +%Y.%m.%d-%H.%M.%S')
COMBINED_FILE="${NET_CODE}.${STATION_CODE}-${DATE_STRING}.mseed"

echo "Combining channels into ${COMBINED_FILE}..."

if ls "${CHANNELS[@]}/%.mseed" 1> /dev/null 2>&1; then
    cat "${CHANNELS[@]}/%.mseed" > "${COMBINED_FILE}"
    echo "Combined file created."
else
    echo "Warning: One or more component files missing, skipping
combine."
fi
else
    echo "Channel combination disabled. Individual files retained."
fi

```

Usage:

- bash must be available
- wget is required for HTTP downloads.
- Network access to the device must be allowed.

1.1.3. Download via Python Script

A Python script is provided for time-based downloads.

The script retrieves data for a specified time window and saves each channel as a miniSEED file. All parameters can be passed as command-line arguments, allowing the script to be used in automated workflows.

Example Script:

```
#!/usr/bin/env python3
"""
```

Time-Based Data Download Script

This script retrieves miniSEED data from a device over HTTP for a specified time window. Data is downloaded per channel and saved to a timestamped output directory.

Usage:

```
python3 time_download.py <device_ip> <date> <start_hour>
<duration_hours>
```

Example:

```
python3 time_download.py 192.168.0.10 2025-02-21 14 1
```

Optional arguments:

```
--net      Network code (default: XX)
--station   Station code (default: XXXX)
--loc       Location code (default: 00)
--channels  Comma-separated channel list (default: HNZ,HNN,HNE)
--combine   Combine all channels into a single miniSEED file
--output    Output directory (default: current directory)
```

Notes:

- Network, station, location, and channel codes must match the device configuration

- These values can be obtained from the device web interface

"""

```
import argparse
```

```
import datetime
```

```
import os
```

```
import sys
```

```
import urllib.request
```

```
#####
```

```
# Helper Functions
```

```
#####
```

```
def to_unix_timestamp(date_str, hour):
```

```
    try:
```

```
        dt = datetime.datetime.strptime(f"{date_str} {hour}", "%Y-%m-%d %H")
```

```
        return int(dt.timestamp())
```

```
    except ValueError:
```

```
        print("Error: Invalid date or hour format. Use YYYY-MM-DD and 0-23.")
```

```
        sys.exit(1)
```

```

def download_file(url, output_path):
    try:
        print(f"Downloading: {url}")
        urllib.request.urlretrieve(url, output_path)
        print(f"Saved to: {output_path}")
    except Exception as e:
        print(f"Error downloading {url}")
        print(e)

#####

# Main
#####

def main():
    parser = argparse.ArgumentParser(
        description="Time-based data download from device (miniSEED)"
    )

    # Required Arguments
    parser.add_argument("ip", help="Device IP address")
    parser.add_argument("date", help="Date (YYYY-MM-DD)")
    parser.add_argument("start_hour", type=int, help="Start hour (0-23)")
    parser.add_argument("hours", type=int, help="Number of hours")

    # Add default values or add with flag
    parser.add_argument("--net", default="XX", help="Network code")
    parser.add_argument("--station", default="XXXX", help="Station code")
    parser.add_argument("--loc", default="00", help="Location code")

    parser.add_argument(
        "--channels",
        default="HNZ,HNN,HNE",
        help="Comma-separated channel list"
    )

```



```

# Optional, combines all downloads to a single file
parser.add_argument(
    "--combine",
    action="store_true",
    help="Combine channels into one file",
)

# Specify output directory for where to save downloads
parser.add_argument(
    "--output",
    default=".",
    help="Base output directory (default: current directory)",
)

args = parser.parse_args()
channels = [ch.strip() for ch in args.channels.split(",") if ch.strip()]

#####
# Time Conversion
#####
start_unix = to_unix_timestamp(args.date, args.start_hour)
end_unix = start_unix + (args.hours * 3600)

print(f"Start UNIX: {start_unix}")
print(f"End UNIX: {end_unix}")

#####
# Output Directory
#####
timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
output_dir = os.path.join(
    args.output,
    f"data_{args.ip}_{timestamp}"
)

```

```

os.makedirs(output_dir, exist_ok=True)
print(f"Saving to: {output_dir}")

#####
# Download Channels
#####
downloaded_files = []

for ch in channels:
    filename = f"{ch}.mseed"
    filepath = os.path.join(output_dir, filename)

    url = (
        f"http://{args.ip}/data?"
        f"channel={args.net}.{args.station}.{args.loc}.{ch}"
        f"&from={start_unix}&to={end_unix}"
    )

    download_file(url, filepath)

    if os.path.exists(filepath):
        downloaded_files.append(filepath)

#####
# Combine Files (optional)
#####
if args.combine and downloaded_files:
    dt_string = datetime.datetime.fromtimestamp(start_unix).strftime(
        "%Y.%m.%d-%H.%M.%S"
    )

    combined_name = f"{args.net}.{args.station}-{dt_string}.mseed"
    combined_path = os.path.join(output_dir, combined_name)

    print(f"Combining into: {combined_path}")

```

```

        with open(combined_path, "wb") as outfile:
            for f in downloaded_files:
                with open(f, "rb") as infile:
                    outfile.write(infile.read())

        print("Combined file created.")
    else:
        print("Combine disabled.")

if __name__ == "__main__":
    main()

```

Usage:

- Python 3 must be available
- Network access to the device must be allowed.

1.1.4. Download via MicroSD Card

Data stored on the external microSD card can be accessed by removing the card and inserting it into a microSD card reader on a PC or laptop. The card will appear as a removable drive, allowing files to be copied directly to a local system.

1.1.4.1. Contents of the MicroSD Card

The microSD card contains several categories of files:

System Logs:

- **system.log:** The most recent system logs.
- **status.log:** System state-of-health information, updated periodically.

miniSEED Data:

- **all_miniSEED_files_are_in_here/:** Directory containing miniSEED files for each recorded channel. Filenames include the channel name, sample rate, and a unique counter. Each file corresponds to a continuous recording segment.

Metadata and Auxiliary Files:

- **Dataless:** Metadata for the entire station.
- **XML:** Specific sensor metadata in StationXML format.
- **Calibration and configuration files:** calvals.txt, calib.txt, polezero.txt.

1.1.5. MiniSEED Extractor Tool

The MiniSEED Extractor is designed to address common issues when downloading data from a microSD card:

- **Quick-formatted cards:** On a quick format, miniSEED files are marked as unused, but residual data from previous recordings can remain. Copying these files directly can include old, irrelevant data.
- **Fixed-length file truncation:** miniSEED files on the card are stored in fixed-length blocks. Some recordings may not use the full block, leaving unused space at the end.

The MiniSEED Extractor reads the miniSEED files and creates new output files that:

- Contain only valid, chronological data blocks.
- Truncate any unused or overwritten blocks, preventing inclusion of leftover data.

For full instructions on using the tool, see the Discovery Manual.